WEEK-10 QUESTION BANK

SECTION-I: APPLICATION

1. What are some of the most used applications of Deep Learning?

Deep Learning is used in a variety of fields today. The most used ones are as follows:

- Sentiment Analysis
- Computer Vision
- Automatic Text Generation
- Object Detection
- Natural Language Processing
- Image Recognition

2. What is the role of the Activation functions in Neural Networks?

The reason for using activation functions in Neural Networks are as follows:

1. The idea behind the activation function is to introduce nonlinearity into the neural network so that it can learn more complex functions.

2. Without the Activation function, the neural network behaves as a linear classifier, learning the function which is a linear combination of its input data.

3. The activation function converts the inputs into outputs.

4. The activation function is responsible for deciding whether a neuron should be activated i.e, fired or not.

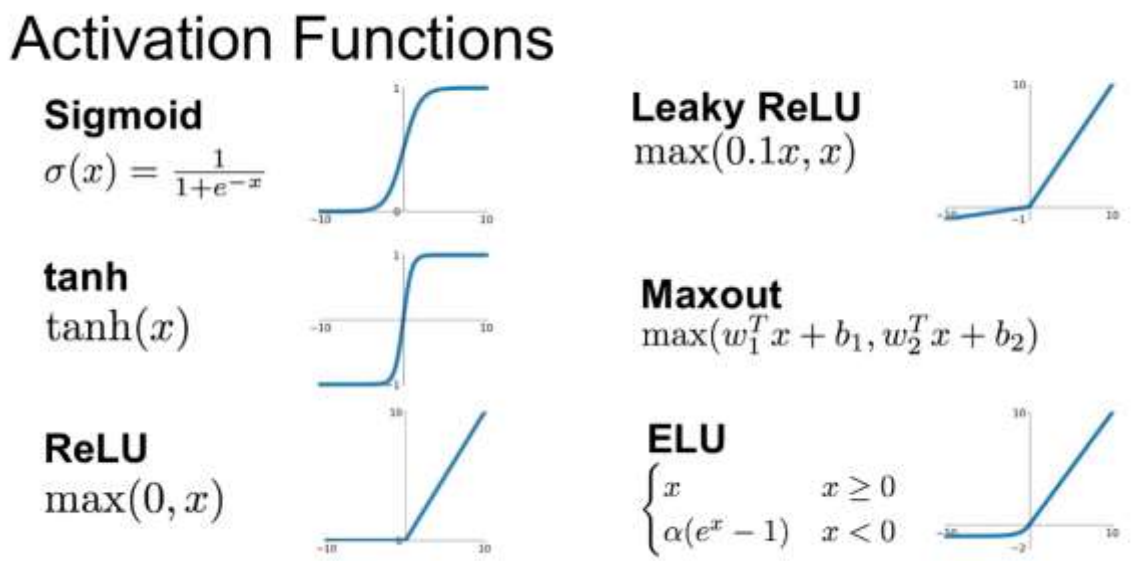
5. To make the decision, firstly it calculates the weighted sum and further adds bias with it.

6. So, the basic purpose of the activation function is to introduce non-linearity into the

3. List down the names of some popular Activation Functions used in Neural Networks.

Some of the popular activation functions that are used while building the deep learning models are as follows:

- Sigmoid function
- Hyperbolic tangent function
- Rectified linear unit (RELU) function
- Leaky RELU function
- Maxout function
- Exponential Linear unit (ELU) function



4. What are the steps involved in training a perception in Deep Learning?

There are five main steps that determine the learning of a perceptron:

1. Initialize thresholds and weights
2. Provide inputs
3. Calculate outputs
4. Update weights in each step
5. Repeat steps 2 to 4

5. What is the use of the loss function?

The loss function is used as a measure of accuracy to see if a neural network has learned accurately from the training data or not. This is done by comparing the training dataset to the testing dataset.

The loss function is a primary measure of the performance of the neural network. In Deep Learning, a good performing network will have a low loss function at all times when training.

6. What are some of the Deep Learning frameworks or tools that you have used?

This question is quite common in a Deep Learning interview. Make sure to answer based on the experience you have with the tools.

However, some of the top Deep Learning frameworks out there today are:

- TensorFlow
- Keras
- PyTorch
- Caffe2

- CNTK
- MXNet
- Theano

7. What are the steps to be followed to use the gradient descent algorithm?

There are five main steps that are used to initialize and use the gradient descent algorithm:

- Initialize biases and weights for the network
- Send input data through the network (the input layer)
- Calculate the difference (the error) between expected and predicted values
- Change values in neurons to minimize the loss function
- Multiple iterations to determine the best weights for efficient working

8. How can hyperparameters be trained in neural networks?

Hyperparameters can be trained using four components as shown below:

- Batch size: This is used to denote the size of the input chunk. Batch sizes can be varied and cut into sub-batches based on the requirement.
- Epochs: An epoch denotes the number of times the training data is visible to the neural network so that it can train. Since the process is iterative, the number of epochs will vary based on the data.
- Momentum: Momentum is used to understand the next consecutive steps that occur with the current data being executed at hand. It is used to avoid oscillations when training.

- Learning rate: Learning rate is used as a parameter to denote the time required for the network to update the parameters and learn.

9. What is the meaning of dropout in Deep Learning?

Dropout is a technique that is used to avoid overfitting a model in Deep Learning. If the dropout value is too low, then it will have minimal effect on learning. If it is too high, then the model can under-learn, thereby, causing lower efficiency.

10. What is the meaning of model capacity in Deep Learning?

In Deep Learning, model capacity refers to the capacity of the model to take in a variety of mapping functions. Higher model capacity means a large amount of information can be stored in the network.

We will check out neural network interview questions alongside as it is also a vital part of Deep Learning.

11. What are some of the advantages of using TensorFlow?

TensorFlow has numerous advantages, and some of them are as follows:

- High amount of flexibility and platform independence
- Trains using CPU and GPU
- Supports auto differentiation and its features
- Handles threads and asynchronous computation easily
- Open-source
- Has a large community

12. What is a CNN?

CNNs are convolutional neural networks that are used to perform analysis on images and visuals. These classes of neural networks can input a multi-channel image and work on it easily.

These Deep Learning questions must be answered in a concise way. So make sure to understand them and revisit them if necessary.

13. What are the various layers present in a CNN?

There are four main layers that form a convolutional neural network:

- Convolution: These are layers consisting of entities called filters that are used as parameters to train the network.
- ReLu: It is used as the activation function and is always used with the convolution layer.
- Pooling: Pooling is the concept of shrinking the complex data entities that form after convolution and is primarily used to maintain the size of an image after shrinkage.
- Connectedness: This is used to ensure that all of the layers in the neural network are fully connected and activation can be computed using the bias easily.

14. What is an RNN in Deep Learning?

RNNs stand for recurrent neural networks, which form to be a popular type of artificial neural network. They are used to process sequences of data, text, genomes, handwriting, and more. RNNs make use of backpropagation for the training requirements.

15. What is a vanishing gradient when using RNNs?

Vanishing gradient is a scenario that occurs when we use RNNs. Since RNNs make use of backpropagation, gradients at every step of the way will tend to get smaller as the network traverses through backward iterations. This equates to the model learning very slowly, thereby, causing efficiency problems in the network.

16. What is exploding gradient descent in Deep Learning?

Exploding gradients are an issue causing a scenario that clumps up the gradients. This creates a large number of updates of the weights in the model when training.

The working of gradient descent is based on the condition that the updates are small and controlled. Controlling the updates will directly affect the efficiency of the model.

17. **Why is the Leaky ReLU function used in Deep Learning?**

Leaky ReLU, also called LReL, is used to manage a function to allow the passing of small-sized negative values if the input value to the network is less than zero.

18. What are some of the examples of supervised learning algorithms in Deep Learning?

There are three main supervised learning algorithms in Deep Learning:

- Artificial neural networks
- Convolutional neural networks
- Recurrent neural networks

19. What are some of the examples of unsupervised learning algorithms in Deep Learning?

There are three main unsupervised learning algorithms in Deep Learning:

- Autoencoders
- Boltzmann machines
- Self-organizing maps

20. Can we initialize the weights of a network to start from zero?

Yes, it is possible to begin with zero initialization. However, it is not recommended to use because setting up the weights to zero initially will cause all of the neurons to produce the same output and the same gradients when performing backpropagation. This means that the network will not have the ability to learn at all due to the absence of asymmetry between each of the neurons.

21.What do you mean by Cost Function?

While building deep learning models, our whole objective is to minimize the cost function.

A cost function explains how well the neural network is performing for its given training data and the expected output.

It may depend on the neural network parameters such as weights and biases. As a whole, it provides the performance of a neural network.

22. Explain the term “Data Normalization”.

Data normalization is an essential preprocessing step, which is used to rescale the initial values to a specific range. It ensures better convergence during backpropagation.

In general, data normalization boils down each of the data points to subtracting the mean and dividing by its standard deviation. This technique improves the performance and stability of neural networks since we normalized the inputs in every layer.

23. What is the difference between Forward propagation and Backward Propagation in Neural Networks?

Forward propagation: The input is fed into the network. In each layer, there is a specific activation function and between layers, there are weights that represent the connection strength of the neurons. The input runs through the individual layers of the network, which ultimately generates an output.

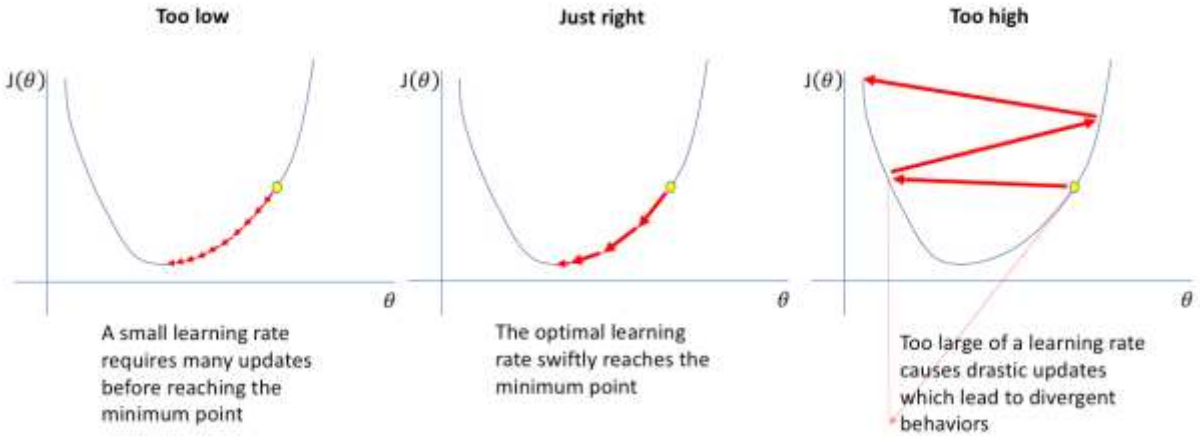
Backward propagation: an error function measures how accurate the output of the network is. To improve the output, the weights have to be optimized. The backpropagation algorithm is used to determine how the individual weights have to be adjusted. The weights are adjusted during the gradient descent method.

24. How does the learning rate affect the training of the Neural Network?

While selecting the learning rate to train the neural network, we have to choose the value very carefully due to the following reasons:

If the learning rate is set too low, training of the model will continue very slowly as we are making very small changes to the weights since our step size that is governed by the equation of gradient descent is small. It will take many iterations before reaching the point of minimum loss.

If the learning rate is set too high, this causes undesirable divergent behavior to the loss function due to large changes in weights due to a larger value of step size. It may fail to converge (the model can give a good output) or even diverge (data is too chaotic for the network to train).



25. What do you mean by Hyperparameters in Neural Networks?

Once the data is formatted correctly, we are usually working with hyperparameters in neural networks. A hyperparameter is a kind of parameter whose values are fixed before the learning process begins.

It decides how a neural network is trained and also the structure of the network which includes:

- The number of hidden units
- The learning rate
- The number of epochs, etc.

26. Why is ReLU the most commonly used Activation Function?

ReLU (Rectified Linear Unit) is the most commonly used activation function in neural networks due to the following reasons:

1. No vanishing gradient: The derivative of the RELU activation function is either 0 or 1, so it could be not in the range of [0,1]. As a result, the product of several derivatives would also be either 0 or 1, because of this property, the vanishing gradient problem doesn't occur during backpropagation.

2. Faster training: Networks with RELU tend to show better convergence performance. Therefore, we have a much lower run time.

3. Sparsity: For all negative inputs, a RELU generates an output of 0. This means that fewer neurons of the network are firing. So we have sparse and efficient activations in the neural network.

27. Explain the vanishing and exploding gradient problems.

These are the major problems in training deep neural networks.

While Backpropagation, in a network of n hidden layers, n derivatives will be multiplied together. If the derivatives are large **e.g, If use ReLU like activation function** then the value of the gradient will increase exponentially as we propagate down the model until they eventually explode, and this is what we call the problem of Exploding gradient.

On the contrary, if the derivatives are small **e.g, If use a Sigmoid activation function** then the gradient will decrease exponentially as we propagate through the model until it eventually vanishes, and this is the Vanishing gradient problem.

28. What do you mean by Optimizers?

Optimizers are algorithms or methods that are used to adjust the parameters of the neural network such as weights, biases, and learning rate, etc to minimize the loss function. These are used to solve the optimization problems by minimizing the function.

The most common used optimizers in deep learning are as follows:

- Gradient Descent
- Stochastic Gradient Descent (SGD)
- Mini Batch Stochastic Gradient Descent (MB-SGD)
- SGD with momentum
- Nesterov Accelerated Gradient (NAG)
- Adaptive Gradient (AdaGrad)
- AdaDelta
- RMSprop
- Adam

29. What is the difference between Epoch, Batch, and Iteration in Neural Networks?

Epoch, iteration, and batch are different types that are used for processing the datasets and algorithms for gradient descent. All these three methods, i.e., epoch, iteration, and batch size are basically ways of working on the gradient descent depending on the size of the data set.

Epoch: It represents one iteration over the entire training dataset (everything put into the training model).

Batch: This refers to when we are not able to pass the entire dataset into the neural network at once due to the problem of high computations, so we divide the dataset into several batches.

Iteration: Let’s have 10,000 images as our training dataset and we choose a batch size of 200. then an epoch should run (10000/200) iterations i.e, 50 iterations.

30. Explain the examples of data processing in Keras.

Answer: Some of the examples include: Firstly, neural networks don’t process raw data, like text files, encoded JPEG image files, or CSV files. They process vectorized & standardized representations. Secondly, text files need to be read into string tensors, then split into words. Finally, the words need to be indexed and turned into integer tensors. Thirdly, images need to be read and decoded into integer tensors, then converted to floating points and normalized to small values (usually between 0 and 1). Lastly, CSV data needs to be parsed, with numerical features converted to floating-point tensors and categorical features indexed and converted to integer tensors. Then each feature typically needs to be normalized to zero-mean and unit variance.

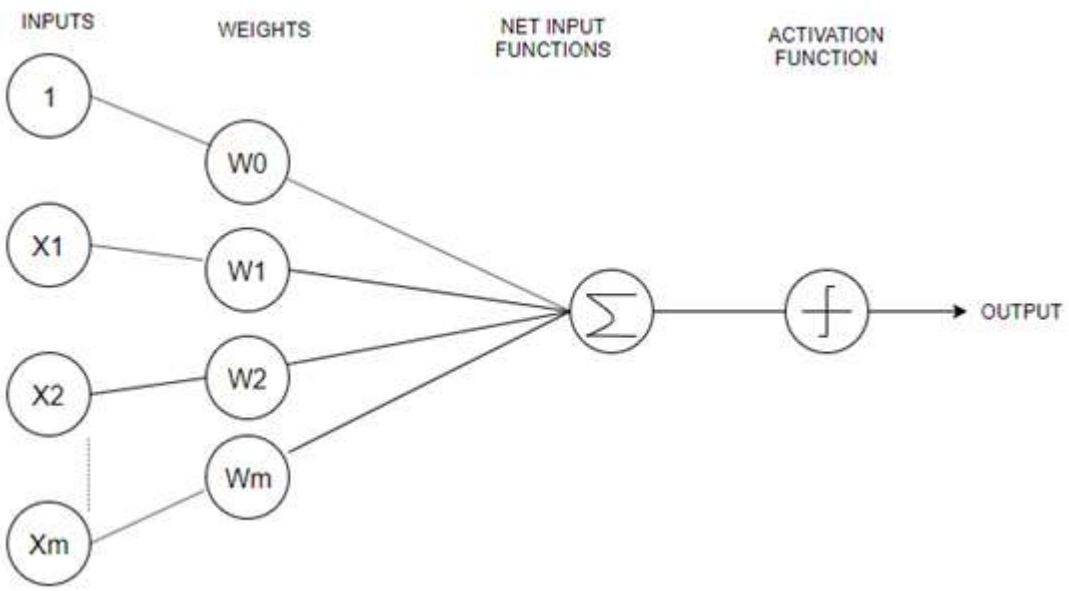
31. Name the types of inputs in the Keras model.

Answer: Keras models accept three types of inputs: Firstly, NumPy arrays, just like Scikit-Learn and many other Python-based libraries. This is a good option if your data fits in memory. Secondly, TensorFlow Dataset objects. This is a high-performance option that is more suitable for datasets that do not fit in memory and that are streamed from a disk or from a distributed filesystem. Lastly, Python generators that yield batches of data (such as custom subclasses of the keras.utils.Sequence class).

32. What do you mean by Perceptron?

A perceptron also called an artificial neuron is a neural network unit that does certain comput ations to detect features.

It is a single-layer neural network used as a linear classifier while working with a set of input data. Since perceptron uses classified data points which are already labeled, it is a supervised learning algorithm. This algorithm is used to enable neurons to learn and process elements in the training set one at a time.



33. What are the different types of Perceptrons?

There are two types of perceptrons:

1. Single-Layer Perceptrons

Single-layer perceptrons can learn only linearly separable patterns.

2. Multilayer Perceptrons

Multilayer perceptrons, also known as feedforward neural networks having two or more layers have a higher processing power.

3. What is the use of the Loss functions?

The loss function is used as a measure of accuracy to identify whether our neural network has learned the patterns accurately or not with the help of the training data.

This is completed by comparing the training data with the testing data.

Therefore, the loss function is considered as a primary measure for the performance of the neural network. In Deep Learning, a good-performing neural network will have a low value of the loss function at all times when training happens.

34. What is the role of the Activation functions in Neural Networks?

The reason for using activation functions in Neural Networks are as follows:

1. The idea behind the activation function is to introduce nonlinearity into the neural network so that it can learn more complex functions.

2. Without the Activation function, the neural network behaves as a linear classifier, learning the function which is a linear combination of its input data.

3. The activation function converts the inputs into outputs.

4. The activation function is responsible for deciding whether a neuron should be activated i.e, fired or not.

5. To make the decision, firstly it calculates the weighted sum and further adds bias with it.

6. So, the basic purpose of the activation function is to introduce non-linearity into the output of a neuron.

5. List down the names of some popular Activation Functions used in Neural Networks.

Some of the popular activation functions that are used while building the deep learning models are as follows:

- Sigmoid function
- Hyperbolic tangent function
- Rectified linear unit (RELU) function
- Leaky RELU function
- Maxout function
- Exponential Linear unit (ELU) function

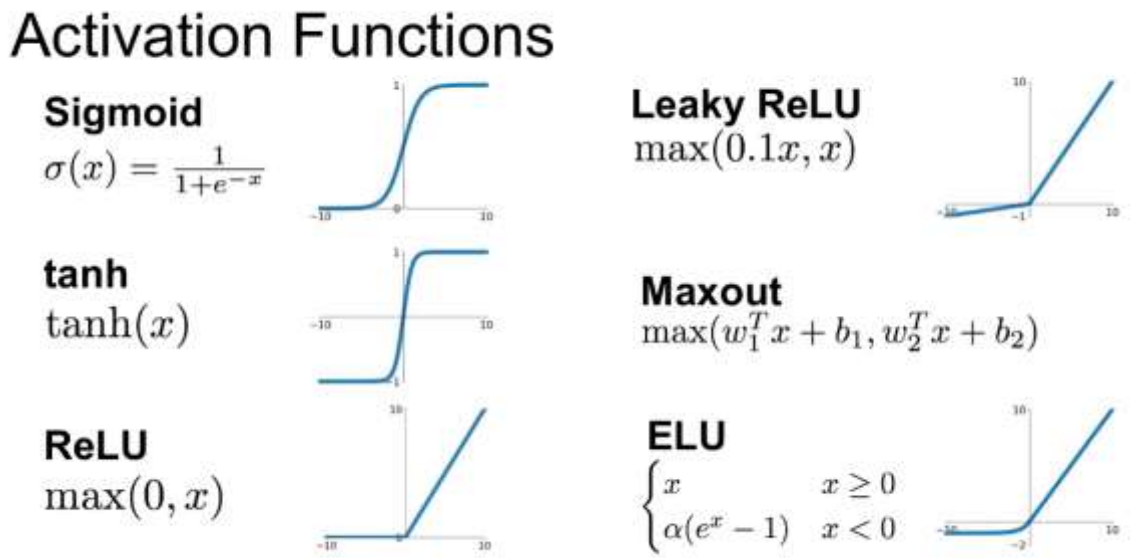


Image Source: Google Images

35. What do you mean by Cost Function?

While building deep learning models, our whole objective is to minimize the cost function.

A cost function explains how well the neural network is performing for its given training data and the expected output.

It may depend on the neural network parameters such as weights and biases. As a whole, it provides the performance of a neural network.

36. What do you mean by Backpropagation?

The backpropagation algorithm is used to train multilayer perceptrons. It propagates the error information from the end of the network to all the weights inside the network. It allows the efficient computation of the gradient or derivatives.

Backpropagation can be divided into the following steps:

- It can forward the propagation of training data through the network to generate output.
- It uses target value and output value to compute error derivatives by concerning the output activations.
- It can backpropagate to calculate the derivatives of the error concerning output activations in the previous layer and continue for all the hidden layers.
- It uses the previously computed derivatives for output and all hidden layers to calculate the error derivative concerning weights.
- It updates the weights and repeats until the cost function is minimized.

37. How to initialize Weights and Biases in Neural Networks?

Neural network initialization means initialized the values of the parameters i.e, weights and biases. Biases can be initialized to zero but we can’t initialize weights with zero.

Weight initialization is one of the crucial factors in neural networks since bad weight initialization can prevent a neural network from learning the patterns.

On the contrary, a good weight initialization helps in giving a quicker convergence to the global minimum. As a rule of thumb, the rule for initializing the weights is to be close to zero without being too small.

38. Why is zero initialization of weight, not a good initialization technique?

If we initialize the set of weights in the neural network as zero, then all the neurons at each layer will start producing the same output and the same gradients during backpropagation.

As a result, the neural network cannot learn anything at all because there is no source of asymmetry between different neurons. Therefore, we add randomness while initializing the weight in neural networks.

39. Explain Gradient Descent and its types.

Gradient Descent is an optimization algorithm that aims to minimize the cost function or to minimize an error. Its main objective is to find the local or global minima of a function based on its convexity. This determines in which direction the model should go to reduce the error.

There are three types of gradient descent:

- Mini-Batch Gradient Descent
- Stochastic Gradient Descent
- Batch Gradient Descent

40. Explain the different steps used in Gradient Descent Algorithm.

The five main steps that are used to initialize and use the gradient descent algorithm are as follows:

- Initialize biases and weights for the neural network.
- Pass the input data through the network i.e, the input layer.
- Compute the difference or the error between the expected and the predicted values.
- Adjust the values i.e, weight updation in neurons to minimize the loss function.

- We repeat the same steps i.e, multiple iterations to determine the best weights for efficient working.

41. Explain the term “Data Normalization”.

Data normalization is an essential preprocessing step, which is used to rescale the initial values to a specific range. It ensures better convergence during backpropagation.

In general, data normalization boils down each of the data points to subtracting the mean and dividing by its standard deviation. This technique improves the performance and stability of neural networks since we normalized the inputs in every layer.

42. What is the difference between Forward propagation and Backward Propagation in Neural Networks?

Forward propagation: The input is fed into the network. In each layer, there is a specific activation function and between layers, there are weights that represent the connection strength of the neurons. The input runs through the individual layers of the network, which ultimately generates an output.

Backward propagation: an error function measures how accurate the output of the network is. To improve the output, the weights have to be optimized. The backpropagation algorithm is used to determine how the individual weights have to be adjusted. The weights are adjusted during the gradient descent method.

43. Explain the different types of Gradient Descent in detail.

Stochastic Gradient Descent: In Stochastic gradient descent, a batch size of 1 is used. As a result, we get n batches. Therefore, the weights of the neural networks are updated after each training sample.

Mini-batch Gradient Descent: In Mini-batch Gradient Descent, the batch size must be between 1 and the size of the training dataset. As a result, we get k batches. Therefore, the weights of the neural networks are updated after each mini-batch iteration.

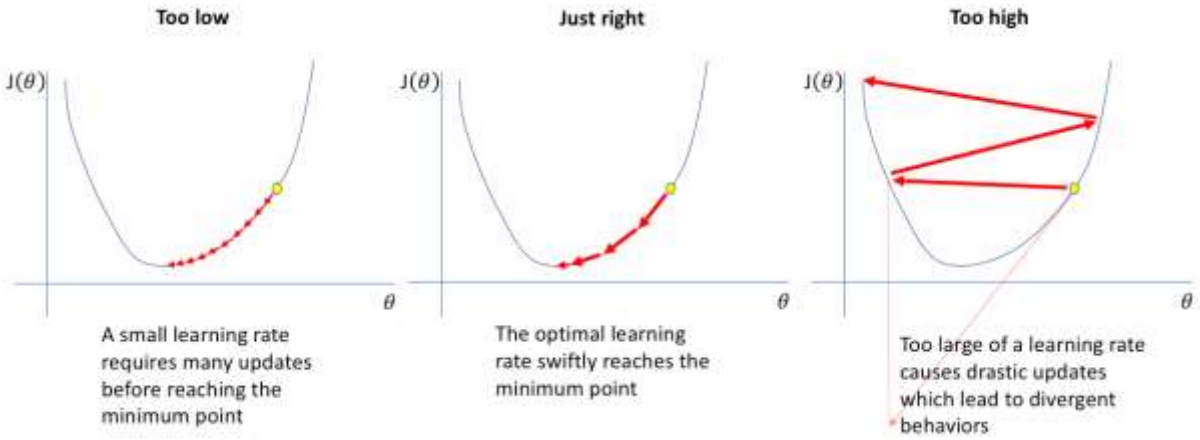
Batch Gradient Descent: In Batch Gradient Descent, the batch size is equal to the size of the training dataset. Therefore, the weights of the neural network are updated after each epoch.

44. How does the learning rate affect the training of the Neural Network?

While selecting the learning rate to train the neural network, we have to choose the value very carefully due to the following reasons:

If the learning rate is set too low, training of the model will continue very slowly as we are making very small changes to the weights since our step size that is governed by the equation of gradient descent is small. It will take many iterations before reaching the point of minimum loss.

If the learning rate is set too high, this causes undesirable divergent behavior to the loss function due to large changes in weights due to a larger value of step size. It may fail to converge (the model can give a good output) or even diverge (data is too chaotic for the network to train).



45. What do you mean by Hyperparameters?

Once the data is formatted correctly, we are usually working with hyperparameters in neural networks. A hyperparameter is a kind of parameter whose values are fixed before the learning process begins.

It decides how a neural network is trained and also the structure of the network which includes:

- The number of hidden units
- The learning rate
- The number of epochs, etc.

46. Why is ReLU the most commonly used Activation Function?

ReLU (Rectified Linear Unit) is the most commonly used activation function in neural networks due to the following reasons:

1. No vanishing gradient: The derivative of the RELU activation function is either 0 or 1, so it could be not in the range of [0,1]. As a result, the product of several derivatives would also be either 0 or 1, because of this property, the vanishing gradient problem doesn't occur during backpropagation.

2. Faster training: Networks with RELU tend to show better convergence performance. Therefore, we have a much lower run time.

3. Sparsity: For all negative inputs, a RELU generates an output of 0. This means that fewer neurons of the network are firing. So we have sparse and efficient activations in the neural network.

47. Explain the vanishing and exploding gradient problems.

These are the major problems in training deep neural networks.

While Backpropagation, in a network of n hidden layers, n derivatives will be multiplied together. If the derivatives are large e.g, If use ReLU like activation function then the value of the gradient will increase exponentially as we propagate down the model until they eventually explode, and this is what we call the problem of Exploding gradient.

On the contrary, if the derivatives are small e.g, If use a Sigmoid activation function then the gradient will decrease exponentially as we propagate through the model until it eventually vanishes, and this is the Vanishing gradient problem.

48. What do you mean by Optimizers?

Optimizers are algorithms or methods that are used to adjust the parameters of the neural network such as weights, biases, and learning rate, etc to minimize the loss function. These are used to solve the optimization problems by minimizing the function.

The most common used optimizers in deep learning are as follows:

- Gradient Descent
- Stochastic Gradient Descent (SGD)
- Mini Batch Stochastic Gradient Descent (MB-SGD)
- SGD with momentum
- Nesterov Accelerated Gradient (NAG)
- Adaptive Gradient (AdaGrad)
- AdaDelta
- RMSprop

49. Name the different ways to build a model with Keras that you know

Answer

- **The Sequential model**: the most approachable API, it's basically a Python list. As such, it’s limited to *simple stacks of layers*.
- **The Functional API**, which focuses on *graph-like* model architectures. It represents a nice mid-point between usability and flexibility, and as such, it’s the most commonly used model-building API.
- **Model subclassing**: a *low-level* option where you write everything yourself from scratch. This is ideal if you want full control over every little thing. However, you won’t get access to many built-in Keras features, and you will be more at risk of making mistakes.

50. What are kernel_initializers in Keras and what is their significance?

Answer

The neural network needs to start with some weights and then iteratively update them to better values. The term kernel_initializer is a fancy term for which statistical distribution or function to use for **initializing the weights**. In the case of some statistical distribution defined, Keras will generate numbers from that statistical distribution and use them as starting weights.

For example, in the code model.add(Dense(13, input_dim=13, kernel_initializer='normal', activation='relu')) **normal distribution** will be used to initialise weights. You can use other

functions and distributions like *Glorot Normal*, *Glorot uniform*, etc.

51.What do compile, fit, and predict do in Keras sequential models?

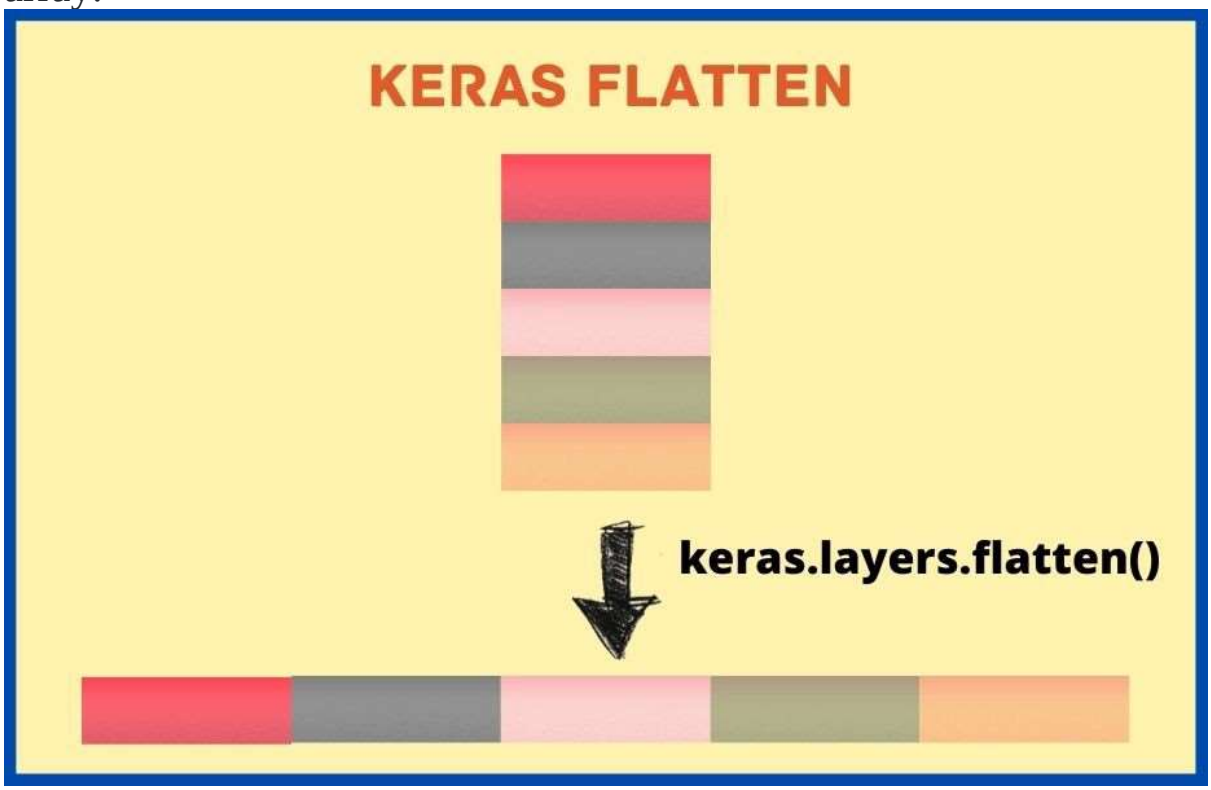
Answer

Those methods perform the steps that we need to do when we want to train a model:

1. First, we want to decide on a **model architecture**, this is the number of *hidden layers* and *activation functions*, and that's what compile does.
2. Secondly, the fit **train our model** to get all the parameters to the correct value to map our *inputs* to our *outputs*.
3. Lastly, we will want to use this model to do some *feed-forward* passes to **predict novel inputs**, that's what the predict method does.

52. What does the Flatten layer do in Keras?

keras.layers.flatten() converts the ***multi-dimensional arrays*** into ***flattened one-dimensional*** arrays or single-dimensional arrays. In other words, it takes all the elements in the original tensor (multi-dimensional array) and puts them into a single-dimensional array.



From the image above we see that the flatten operation on a tensor **reshapes the tensor** to have the shape that is equal to the **number of elements** contained in tensor non including the batch dimension.

53. What is the difference between Keras model.evaluate() and model.predict()?

Answer

- The model.evaluate() function first *predicts* the output for the given input and then *computes* the **metrics function** specified in the model.compile, and based on y_true and y_pred it returns the computed metric value as the output for every batch.
- The model.predict() function will give you the *actual predictions* for all samples in a batch, for all batches.
- So even if you use the same data, the differences between the outputs will be there because the value of a **loss function** will be almost always different than the **predicted values**.

54. What is the difference between predict and predict_on_batch methods of a Keras model?

Answer

The difference lies in when you pass as x data that is **larger than one batch**.

predict will go through all the data, *batch by batch*, predicting labels. It thus internally does the *splitting in batches* and feeding *one batch at a time*.

predict_on_batch, on the other hand, assumes that the data you pass in is *exactly one batch* and thus feeds it to the network. It won't try to split it (which, depending on your setup, might prove problematic for your GPU memory if the array is very big).

55. What's the difference between *units*, *input shape* and *output shape* in a Keras Layer class?

Answer

- The **units** is the amount of *neurons*, or *cells* that the layer has inside it.
- The **shapes** are consequences of the model's configuration and are tuples representing **how many elements an array or tensor has in each dimension**. What flows between layers are tensors. Tensors can be seen as matrices with shapes. In Keras, the **input layer** itself is not a layer, but a **tensor** and its shape it's the **input shape**. It's that **starting tensor** you send to the first hidden layer. This tensor must have the same shape as the training data.
- Given the **input shape**, all other shapes are results of **layers calculations**: The **units** of each layer will define the **output shape** (the shape of the tensor that is produced by the layer and that will be the input of the next layer).
- Furthermore, each type of Layer works in a particular way. For example,
    - Dense layers have output shape based on units
    - Convolutional layers have output shapes based on filters, etc.

56. What is the advantage of using *Functional* vs *Sequential* models in Keras?

Answer

The Sequential model API is a way of creating deep learning models where an instance of the Sequential class is created and model **layers** are created and *added* to it. This is great for developing deep learning models in most situations, but it also has some limitations: For example, it is not straightforward to define models that may have *multiple different input sources*, produce *multiple output destinations* or models that *re-use layers*.

The Keras **functional API** provides a more flexible way for defining models. It specifically allows you to define *multiple input* or *output models* as well as models that *share layers*. More than that, it allows you to define *ad hoc acyclic* network graphs.

In the **functional API**, the models are defined by creating *instances of layers* and connecting them directly to each other in pairs, then defining a Model that specifies the layers to act as the *input* and *output* to the model. As a result, creating complex networks such as *siamese networks* or *residual networks* becomes possible.

57. What are some of the advantages of using TensorFlow?

TensorFlow has numerous advantages, and this is why it is the most used framework for Machine Learning in the world. Some of its advantages are given below:

- Platform independency
- Usage of GPU for distributed computing
- Auto-differentiation capability
- Open-source and large community
- Highly customizable based on requirements

- Support for asynchronous computations

58. Are there any limitations to using TensorFlow?

Even though TensorFlow provides numerous advantages, it has one or two caveats in the current versions:

- No support for OpenCL (Open Computing Language) yet
- GPU memory conflicts when used with Theano
- Can be overwhelming for beginners to get started

Next up on these TensorFlow coding interview questions, let us check out about the types of tensors.

59. What are the types of tensors available in TensorFlow?

There are three main types of tensors:

- Constant tensors
- Variable tensors
- Placeholder tensors

60. How can data be loaded into TensorFlow?

There are two ways that you can use to load data into TensorFlow before training **Machine Learning algorithms**:

- **Data load into memory**: Here, the data is loaded into the memory as a single array unit. It is the easiest way to load the data.
- **TensorFlow data pipeline**: It is making use of the built-in APIs to load the data and feed it across to the algorithm.

61. What is the simple working of an algorithm in TensorFlow?

There are five main steps that govern the working of the majority of algorithms in **TensorFlow**. They are as follows:

1. Data import or data generation, alongside setting up a data pipeline

2. Data input through computational graphs
3. Generation of the loss function to evaluate the output
4. Backpropagation to modify the data
5. Iterating until output criteria are met

62. What are the methods that can be used to handle overfitting in TensorFlow?

There are three methods, which can be used to easily handle the condition of overfitting when using TensorFlow:

- Batch normalization
- Regularization technique
- Dropouts

63. What are placeholder tensors?

Placeholder tensors are entities that provide an advantage over a regular variable. It is used to assign data at a later point in time.

Placeholders can be used to build graphs without any prior data being present. This means that they do not require any sort of initialization for usage.

64. What are managers in TensorFlow?

TensorFlow managers are entities that are responsible for handling the following activities for servable objects:

- Loading
- Unloading
- Lookup
- Lifetime management

65. Where is TensorFlow mostly used?

TensorFlow is used in all of the domains that cover Machine Learning and Deep Learning. Being the most essential tool, the following are some of the main use cases of TensorFlow:

- Time series analysis
- Image recognition
- Voice recognition
- Video upscaling
- Test-based applications

66. What makes TensorFlow advantageous over other libraries?

Following are some of the benefits of TensorFlow over other libraries:

- Pipelines: **data** is used to build efficient pipelines for text and image processing.
- Debugging: **tfdbg** is used to track the state and structure of objects for easy debugging.
- Visualization: TensorBoard provides an elegant user interface for users to visualize graphs.
- Scalability: It can scale Deep Learning applications and their associated infrastructure easily.

67. Differentiate between TensorFlow and PyTorch.

TensorFlow	PyTorch
Developed by Google	Developed by Facebook
No support for runtime graph operations	Provides computational graph operations at runtime
Offers TensorBoard for visualization	No visualization tools offered in the bundle
Based on the Theano library	Based on the Torch library

68. Can you use TensorBoard without installing TensorFlow?

If TensorFlow is not installed, users can still make use of TensorBoard (versions above 1.14) in a standalone mode with redacted features.

Following plugins are supported:

- Scalars
- Image
- Audio
- Graph
- Projector
- Histograms
- Mesh

69. When using TensorFlow, is performance always preferred over accuracy?

No, performance is not always preferred over accuracy when you use TensorFlow. This completely depends on the type of requirement and what the model is trying to achieve. The general rule of thumb is to provide equal weightage to model accuracy and performance.

The next set of TensorFlow interview questions will show the importance of using an example along with concepts to explain.

70. What are some of the important parameters to consider when implementing a random forest algorithm in TensorFlow?

There are six main parameters you should think about and plan when implementing a random forest algorithm in TensorFlow:

- Number of inputs
- Feature count
- Number of samples per batch
- Total number of training steps
- Number of trees
- Maximum number of nodes

71. What are some of the numerical and categorical loss functions supported when working with TensorFlow?

Following are some of the widely used numerical and categorical loss functions supported when working with TensorFlow:

Numerical loss functions:

- L1 loss
- L2 loss
- Pseudo-Huber loss

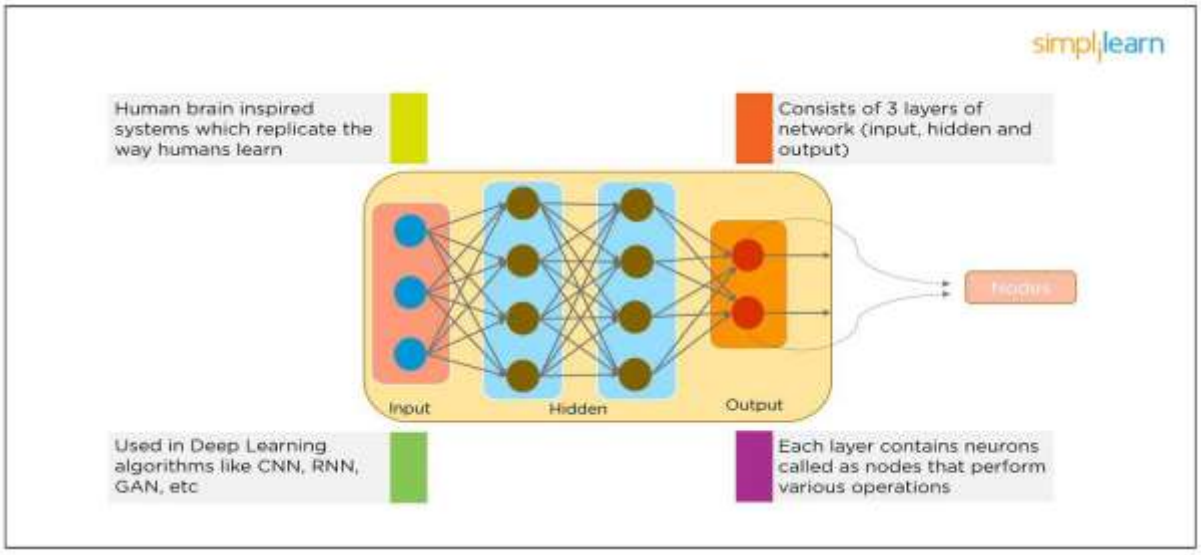
Categorical loss functions:

- Hinge loss
- Cross-entropy loss
- Sigmoid-entropy loss
- Weighted cross-entropy loss

Each of the loss functions mentioned above has a specific use based on the input data and the type of modeling involved.

72. What is a Neural Network?

Neural Networks replicate the way humans learn, inspired by how the neurons in our brains fire, only much simpler.



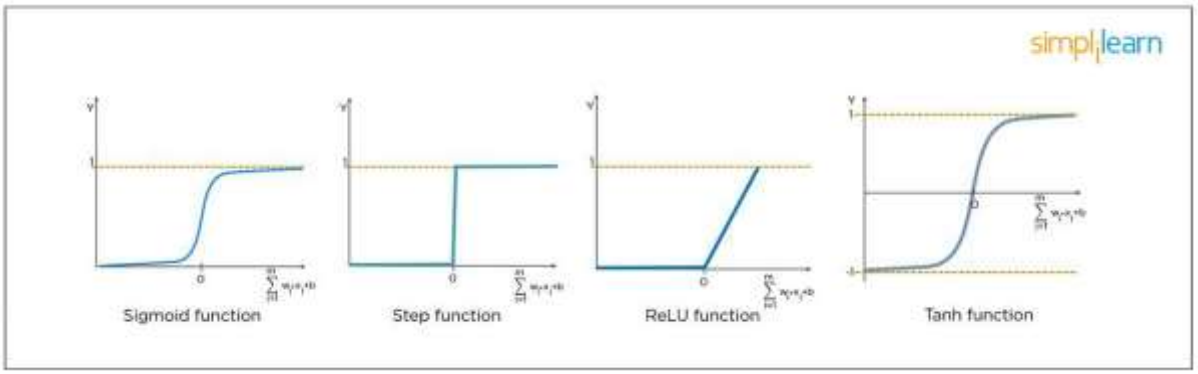
The most common Neural Networks consist of three network layers:

1. An input layer
2. A hidden layer (this is the most important layer where feature extraction takes place, and adjustments are made to train faster and function better)
3. An output layer

Each sheet contains neurons called “nodes,” performing various operations. Neural Networks are used in deep learning algorithms like CNN, RNN, GAN, etc.

73. What Is the Role of Activation Functions in a Neural Network?

At the most basic level, an activation function decides whether a neuron should be fired or not. It accepts the weighted sum of the inputs and bias as input to any activation function. Step function, Sigmoid, ReLU, Tanh, and Softmax are examples of activation functions.



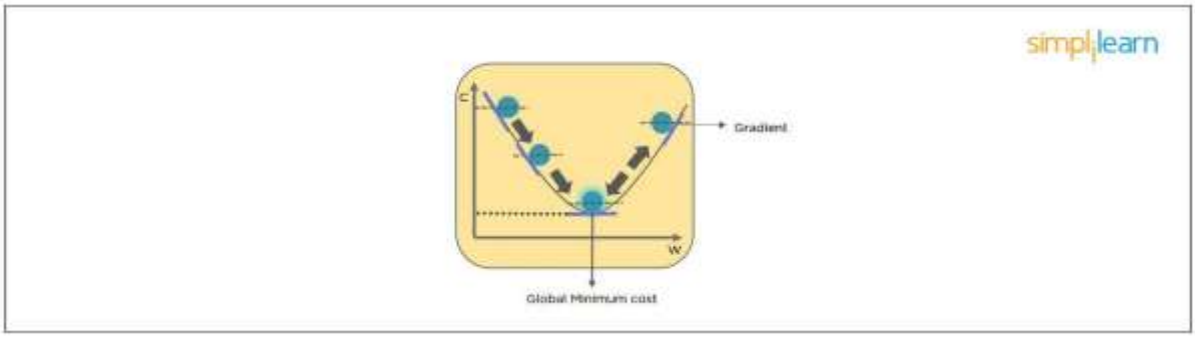
74. What Is the Cost Function?

Also referred to as “loss” or “error,” cost function is a measure to evaluate how good your model’s performance is. It’s used to compute the error of the output layer during backpropagation. We push that error backward through the neural network and use that during the different training functions.



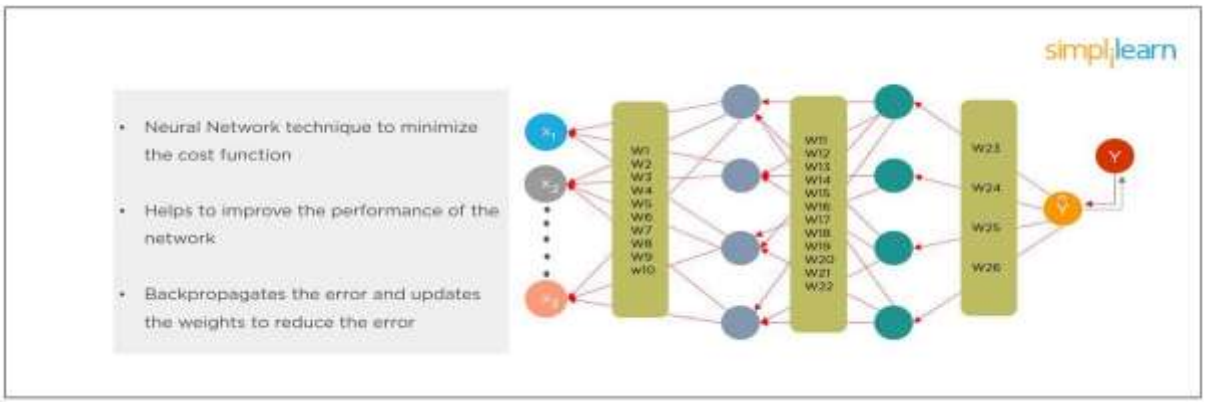
75. What Is Gradient Descent?

Gradient Descent is an optimal algorithm to minimize the cost function or to minimize an error. The aim is to find the local-global minima of a function. This determines the direction the model should take to reduce the error.



76. What Do You Understand by Backpropagation?

This is one of the most frequently asked deep learning interview questions. Backpropagation is a technique to improve the performance of the network. It backpropagates the error and updates the weights to reduce the error.



77. What Is the Difference Between a Feedforward Neural Network and Recurrent Neural Network?

In this deep learning interview question, the interviewee expects you to give a detailed answer.

A Feedforward Neural Network signals travel in one direction from input to output. There are no feedback loops; the network considers only the current input. It cannot memorize previous inputs (e.g., CNN).

A Recurrent Neural Network's signals travel in both directions, creating a looped network. It considers the current input with the previously received inputs for generating the output of a layer and can memorize past data due to its internal memory.



78. What Are the Applications of a Recurrent Neural Network (RNN)?

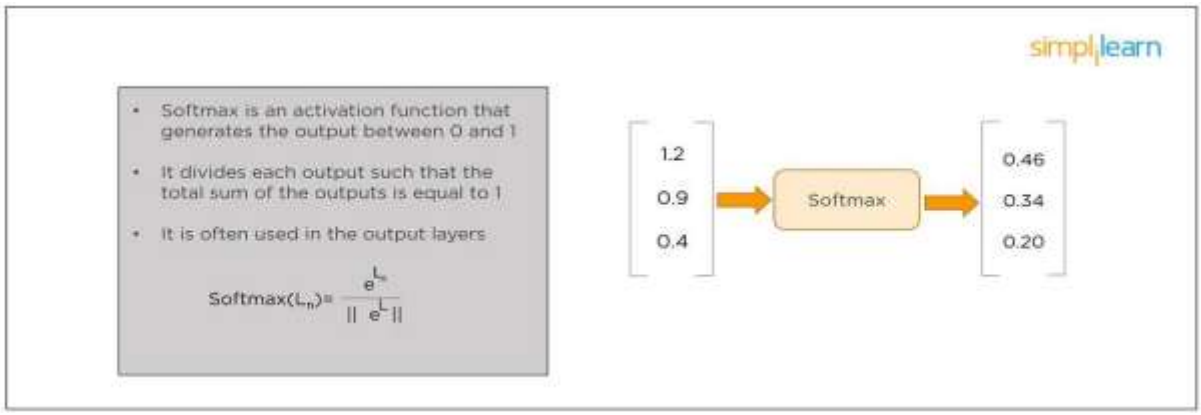
The RNN can be used for sentiment analysis, text mining, and image captioning. Recurrent Neural Networks can also address time series problems such as predicting the prices of stocks in a month or quarter.

PCP in AI and Machine Learning

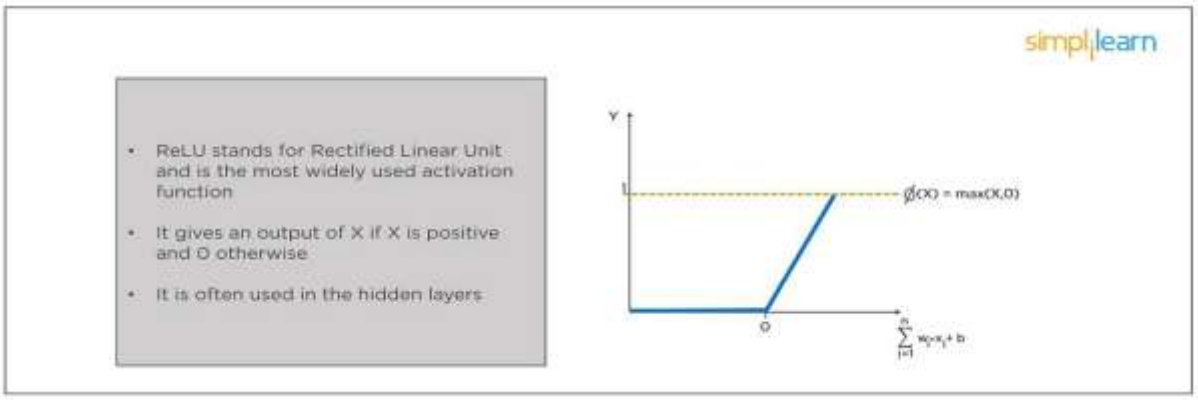
In Partnership with Purdue UniversityEXPLORE COURSE

79. What Are the Softmax and ReLU Functions?

Softmax is an activation function that generates the output between zero and one. It divides each output, such that the total sum of the outputs is equal to one. Softmax is often used for output layers.

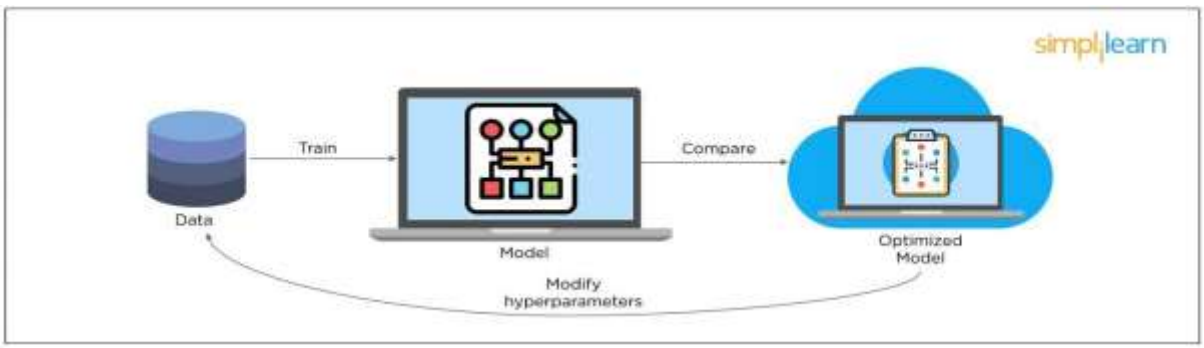


ReLU (or Rectified Linear Unit) is the most widely used activation function. It gives an output of X if X is positive and zeros otherwise. ReLU is often used for hidden layers.



80. What Are Hyperparameters?

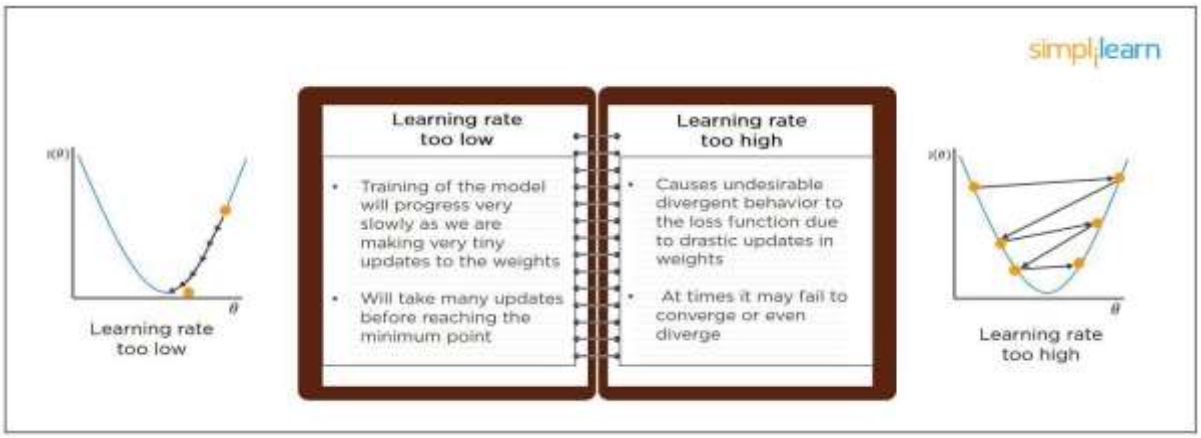
This is another frequently asked deep learning interview question. With neural networks, you're usually working with hyperparameters once the data is formatted correctly. A hyperparameter is a parameter whose value is set before the learning process begins. It determines how a network is trained and the structure of the network (such as the number of hidden units, the learning rate, epochs, etc.).



81. What Will Happen If the Learning Rate Is Set Too Low or Too High?

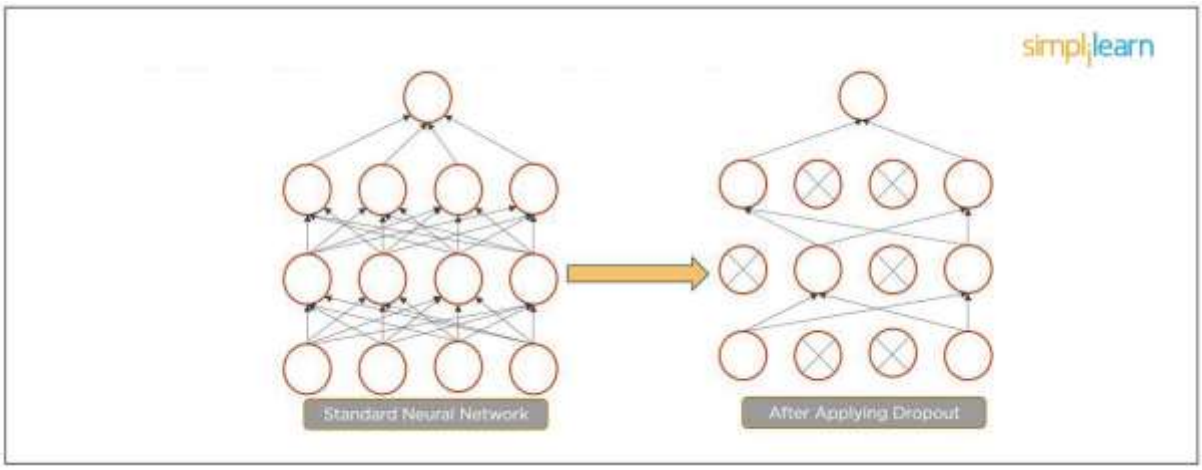
When your learning rate is too low, training of the model will progress very slowly as we are making minimal updates to the weights. It will take many updates before reaching the minimum point.

If the learning rate is set too high, this causes undesirable divergent behavior to the loss function due to drastic updates in weights. It may fail to converge (model can give a good output) or even diverge (data is too chaotic for the network to train).



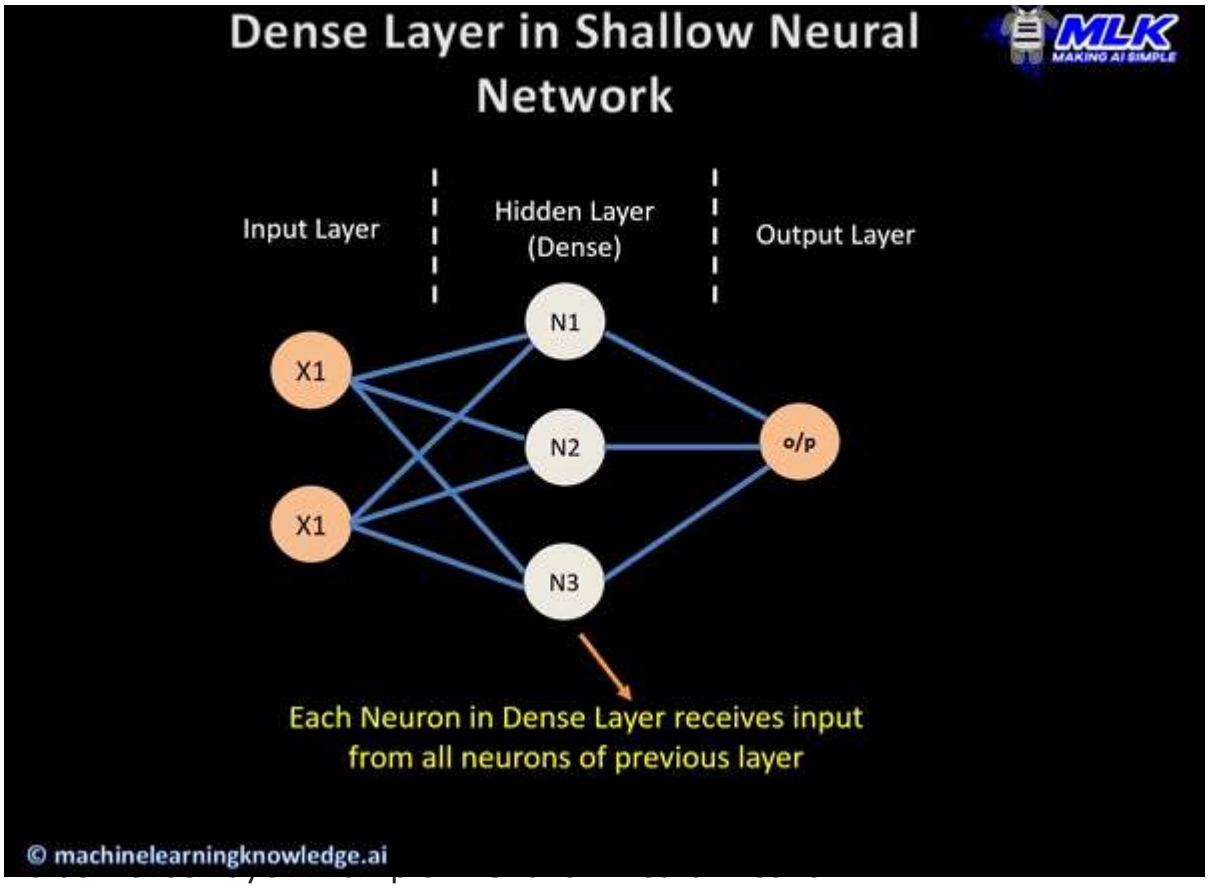
82. What Is Dropout and Batch Normalization?

Dropout is a technique of dropping out hidden and visible units of a network randomly to prevent overfitting of data (typically dropping 20 percent of the nodes). It doubles the number of iterations needed to converge the network.



Batch normalization is the technique to improve the performance and stability of neural networks by normalizing the inputs in every layer so that they have mean output activation of zero and standard deviation of one.

83. How to build Shallow Neural Network with Keras Dense Layer?



Now let's see how a Keras model with a single dense layer is built. Here we are using the in-built Keras Model i.e. Sequential.

First, we provide the input layer to the model and then a dense layer along with ReLU activation is added.

The output layer also contains a dense layer and then we look at the shape of the output of this model.

```
import tensorflow as tf



# Create a `Sequential` model and add a Dense layer as the first layer.

model = tf.keras.models.Sequential()

model.add(tf.keras.Input(shape=(16,)))
```

```
model.add(tf.keras.layers.Dense(32, activation='relu'))



# Now the model will take as input arrays of shape (None, 16)

# and output arrays of shape (None, 32).

# Note that after the first layer, you don't need to specify

# the size of the input anymore:



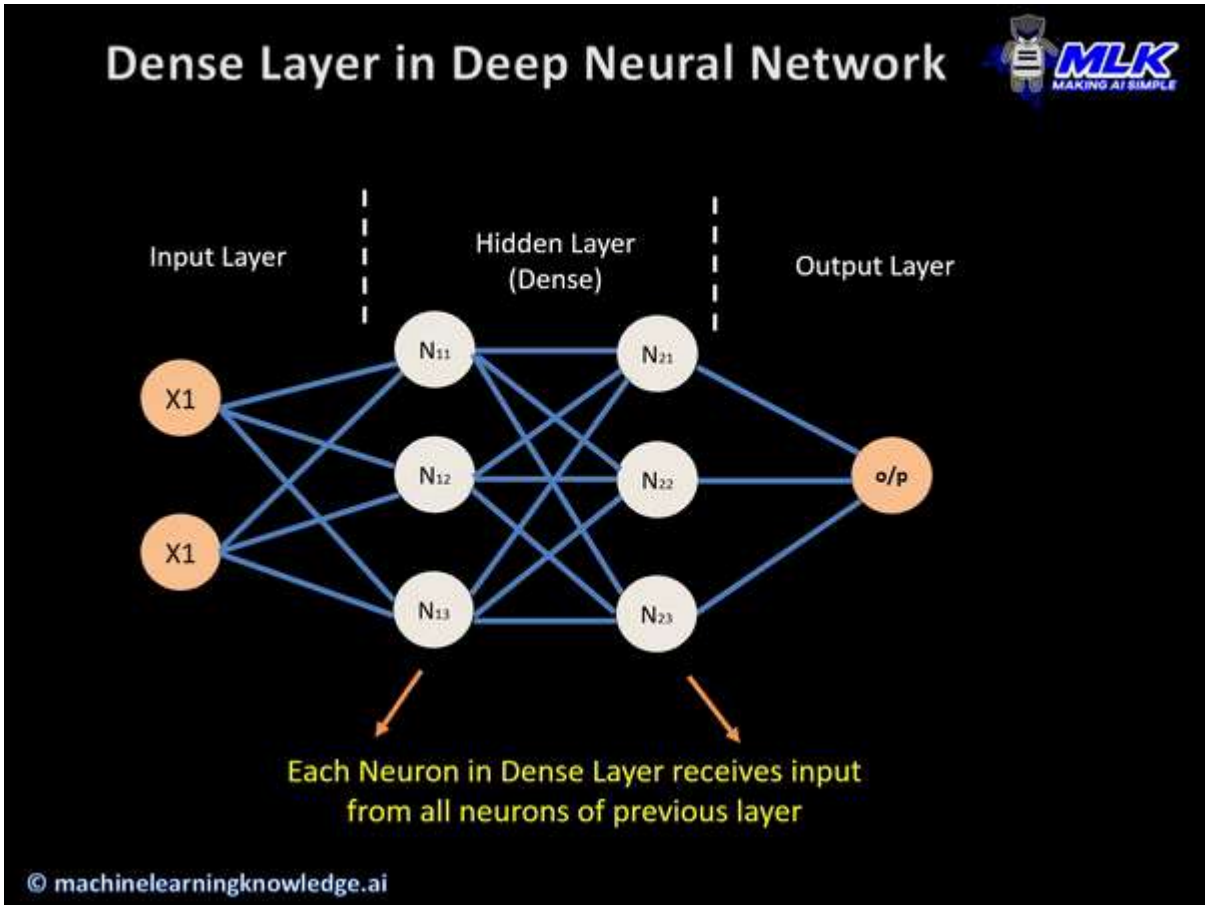
model.add(tf.keras.layers.Dense(32))

model.output_shape
```

Output:

```
(None, 32)
```

84. How to build Deep Neural Network with Keras Dense Layers?



In this example, we look at a model where multiple hidden layers are used in deep neural networks. Here we are using ReLu activation function in the neurons of the hidden dense layer.

Remember one cannot find the weights and summary of the model yet, first the model is provided input data and then we look at the weights present in the model.

At last, the model summary displays the information about the **input layers**, the **shape of output layers,** and the **total count of parameters**.

```
from keras.models import Sequential

from keras.layers import Dense

```

```
model = Sequential(
    [
        Dense(5, activation="relu"),
        Dense(10, activation="relu"),
        Dense(15),
    ]
)  # No weights to be addded here


# Here we cannot check for weights
# model.weights


# Neither we can look at the summary
# model.summary()


# First we must call the model and evaluate it on test data
x = tf.ones((5, 20))
y = model(x)
print("Number of weights after calling the model:",
len(model.weights))
```

Output:

```
Number of weights after calling the model: 6
```

```
# Looking at the Summary of the Model
model.summary()
```

Output:

```
Model: "sequential_2"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
dense_4 (Dense)              (5, 5)                    105
_________________________________________________________________
dense_5 (Dense)              (5, 10)                   60
_________________________________________________________________
dense_6 (Dense)              (5, 15)                   165
=================================================================
Total params: 330
Trainable params: 330
Non-trainable params: 0
_________________________________
```

85. What is a Dense Layer in Neural Network?

The dense layer is a neural network layer that is connected deeply, which means each neuron in the dense layer receives input from all neurons of its previous layer. The dense layer is found to be the most commonly used layer in the models.

In the background, the dense layer performs a matrix-vector multiplication. The values used in the matrix are actually parameters that can be trained and updated with the help of backpropagation.

The output generated by the dense layer is an 'm' dimensional vector. Thus, dense layer is basically used for changing the dimensions of the vector. Dense layers also applies operations like rotation, scaling, translation on the vector.

Syntax

keras.layers.Dense(units, activation=None, use_bias=True, kernel_initializer='glorot_uniform', bias_initializer='zeros', kernel_regularizer=None, bias_regularizer=None, activity_regularizer=None, kernel_constraint=None, bias_constraint=None)

86. What are the different Keras Dense Layer Parameters?

Let us see different parameters of dense layer function of Keras below

1. Units

The **most basic parameter** of all the parameters, it uses positive integer as it value and represents the **output size** of the layer.

It is the unit parameter itself that plays a major role in the **size of the weight matrix** along with the **bias vector**.

2. Activation

The activation parameter is helpful in applying the element-wise activation function in a dense layer. By default, Linear

Activation is used but we can alter and switch to any one of many options that Keras provides for this.

3. Use_Bias

Another straightforward parameter, **use_bias** helps in deciding whether we should include a bias vector for calculation purposes or not. By default, **use_bias** is set to true.

4. Initializers

As its name suggests, the initializer parameter is used for providing input about how values in the layer will be initialized. In case of the Dense Layer, the weight matrix and bias vector has to be initialized.

5. Regularizers

Regularizers contain three parameters that carry out regularization or penalty on the model. Generally, these parameters are not used regularly but they can help in the generalization of the model.

6. Constraints

This last parameter determines the constraints on the values that the weight matrix or bias vector can take.

87. How to implement the Keras Dense Layer Operation?

The dense layer function of Keras implements following operation –

output = activation(dot(input, kernel) + bias)

In the above equation, **activation** is used for performing **element-wise activation** and the **kernel** is the **weights matrix** created by the layer, and **bias** is a bias vector created by the layer.

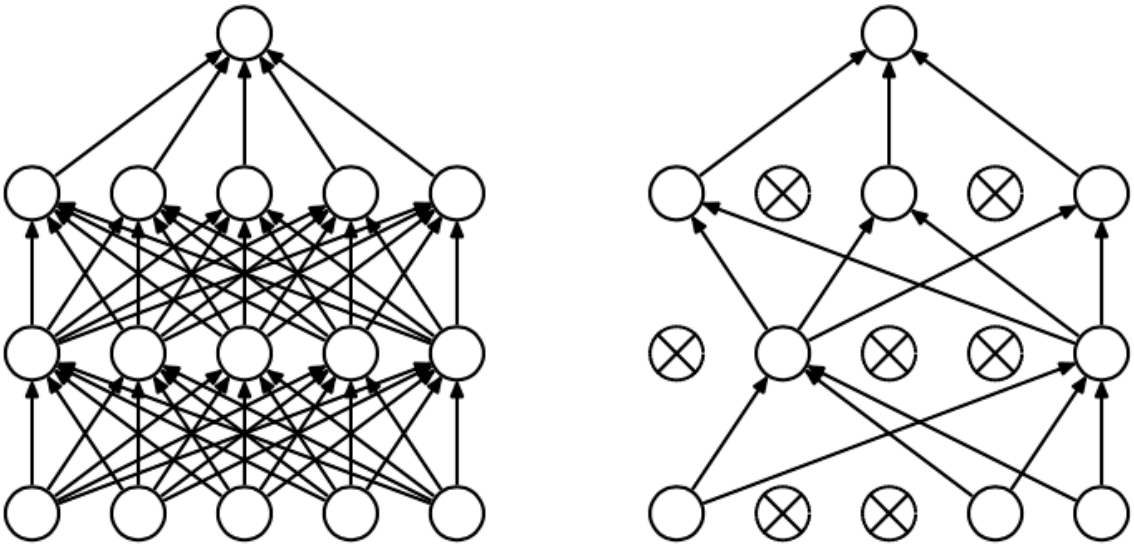
Keras dense layer on the output layer performs **dot product** of **input tensor** and **weight kernel matrix**.

A bias vector is added and element-wise activation is performed on output values.

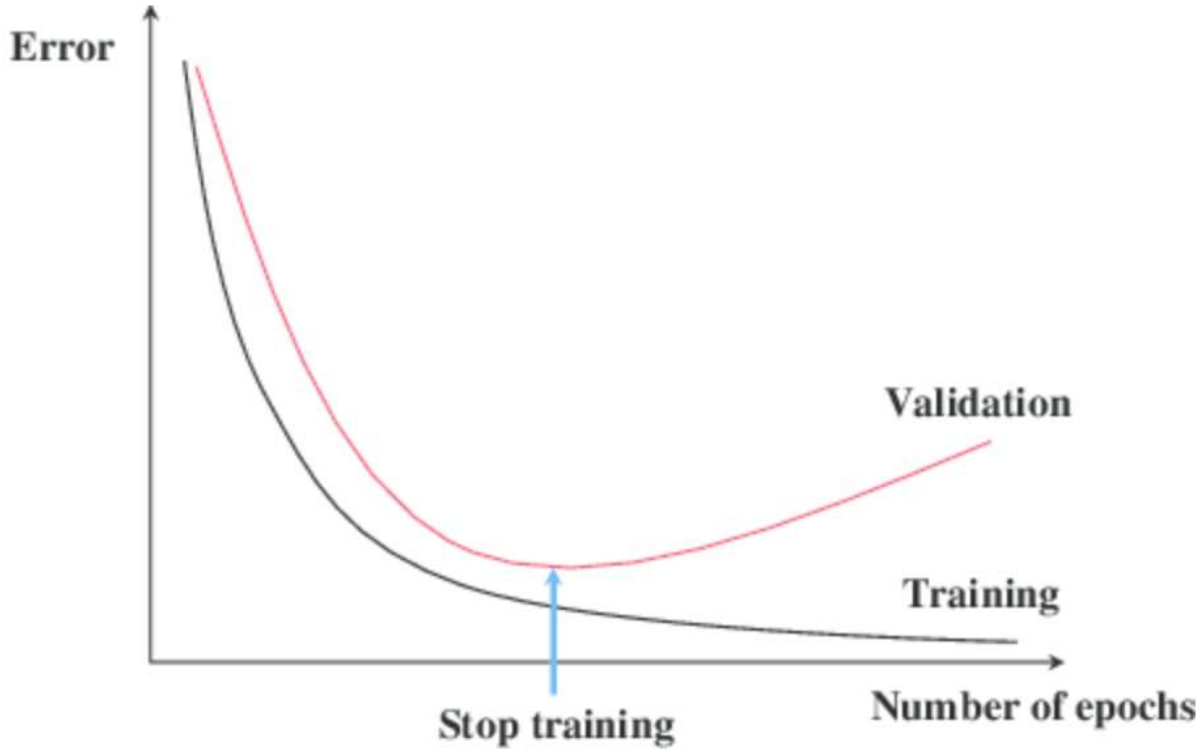
SECTION-II: ANALYSIS

1. Overfitting is one of the most common problems every Machine Learning practitioner faces. Explain some methods to avoid overfitting in Neural Networks.

Dropout: It is a regularization technique that prevents the neural network from overfitting. It randomly drops neurons from the neural network during training which is equivalent to training different neural networks. The different networks will overfit differently, so the net effect of the dropout regularization technique will be to reduce overfitting so that our model will be good for predictive analysis.



Early stopping: This regularization technique updates the model to make it better fit the training data with each iteration. After a certain number of iterations, new iterations improve the model. After that point, however, the model begins to overfit the training data. Early stopping refers to stopping the training process before that point.



2. Suppose we have a perceptron having weights corresponding to the three inputs have the following values:

$w_1 = 2$; $w_2 = -4$; and $w_3 = 1$

and the activation of the unit is given by the step-function:

$\varphi(v) = 1$ if $v \geq 0$ otherwise 0

Calculate the output value y of the given perceptron for each of the following input patterns:

Pattern	P_1	P_2	P_3	P_4
X_1	1	0	1	1
X_2	0	1	0	1
X_3	0	1	1	1

SOLUTION:

To calculate the output value y for each of the given patterns we have to follow below two steps:

a) Calculate the weighted sum: $v = \Sigma_i (w_i x_i) = w_1 \cdot x_1 + w_2 \cdot x_2 + w_3 \cdot x_3$

b) Apply the activation function to v.

The calculations for each input pattern are:

$P_1 : v = 2 \cdot 1 - 4 \cdot 0 + 1 \cdot 0 = 2, (2>0), \ y = \varphi(2) = 1$

$P_2 : v = 2 \cdot 0 - 4 \cdot 1 + 1 \cdot 1 = -3, (-3<0), \ y = \varphi(-3) = 0$

$P_3 : v = 2 \cdot 1 - 4 \cdot 0 + 1 \cdot 1 = 3, (3>0), \ y = \varphi(3) = 1$

$P_4 : v = 2 \cdot 1 - 4 \cdot 1 + 1 \cdot 1 = -1, (-1<0), \ y = \varphi(-1) = 0$

3. Consider a feed-forward Neural Network having 2 inputs(label -1 and label -2)with fully connected layers and we have 2 hidden layers:

Hidden layer-1: Nodes labeled as 3 and 4

Hidden layer-2: Nodes labeled as 5 and 6

A weight on the connection between nodes i and j is represented by w_{ij}, such as w_{24} is the weight on the connection between nodes 2 and 4. The following lists contain all the weights values used in the given network:

$w_{13} = -2, w_{35} = 1, w_{23} = 3, w_{45} = -1, w_{14} = 4, w_{36} = -1, w_{24} = -1, w_{46} = 1$

Each of the nodes 3, 4, 5, and 6 use the following activation function:

$\varphi(v) = 1$ if $v \geq 0$ otherwise 0

where v denotes the weighted sum of a node. Each of the input nodes (1 and 2) can only receive binary values (either 0 or 1). Calculate the output of the network (y_5 and y_6) for the input pattern given by (node-1 and node-2 as 0, 0 respectively).

SOLUTION:

To find the output of the network it is necessary to calculate weighted sums of hidden nodes 3 and 4:

$v_3 = w_{13}x_1 + w_{23}x_2$, $v_4 = w_{14}x_1 + w_{24}x_2$

Then find the outputs from hidden nodes using activation function φ:

$y_3 = \varphi(v_3)$, $y_4 = \varphi(v_4)$.

Use the outputs of the hidden nodes y_3 and y_4 as the input values to the output layer (nodes 5 and 6), and find weighted sums of output nodes 5 and 6:

$v_5 = w_{35}y_3 + w_{45}y_4$, $v_6 = w_{36}y_3 + w_{46}y_4$.

Finally, find the outputs from nodes 5 and 6 (also using φ):

$y_5 = \varphi(v_5)$, $y_6 = \varphi(v_6)$.

The output pattern will be (y_5, y_6).

Perform this calculation for the given input – Input pattern (0, 0)

$v_3 = -2 \cdot 0 + 3 \cdot 0 = 0$, $y_3 = \varphi(0) = 1$

$v_4 = 4 \cdot 0 - 1 \cdot 0 = 0$, $y_4 = \varphi(0) = 1$

$v_5 = 1 \cdot 1 - 1 \cdot 1 = 0$, $y_5 = \varphi(0) = 1$

$v_6 = -1 \cdot 1 + 1 \cdot 1 = 0$, $y_6 = \varphi(0) = 1$

Therefore, the output of the network for a given input pattern is (1, 1).

4. How would you freeze all the layers in the Keras model

Problem

Consider the code:

```
from keras.models import Sequential
from keras.layers import Dense
from tensorflow.keras.optimizers import SGD

# Set model
model=Sequential()
layer=Dense(64,kernel_initializer='glorot_uniform',input_shape=(784,))
model.add(layer)
layer2=Dense(784, activation='sigmoid',kernel_initializer='glorot_uniform')
model.add(layer2)

# Set optimizer
opt = SGD()

# Compile
model.compile(loss='relu', optimizer=opt,metrics = ['mae'])

# Complete code to freeze layers
#  . . .
```

ANSWER:

All layers & models have a layer.trainable boolean attribute. To freeze them, we just iterate over the layers and set the attribute to False:

```
# Loop for freezing all layers
for layer in model.layers:
        layer.trainable = False

# Checking
print(layer.trainable)
print(layer2.trainable)
```

Output:

```
False
False
```

5. How to create custom metrics with Keras?

Answer

You can implement a **custom metric** in two ways:

As simple callables, in which the callable has signature metric_fn(y_true, y_pred) that returns an array of losses (*one of sample in the input batch*). y_true and y_pred must be tensors so in order to correctly calculate the metric you need to use keras.backend functionality. For example:

```
import keras.backend as K

def mean_pred(y_true, y_pred):
    return K.mean(y_pred)

model.compile(optimizer='sgd',
              loss='binary_crossentropy',
              metrics=['accuracy', mean_pred])
```

As subclasses of Metric, in which you have to override the update_state, result, and reset_state functions, where:

update_state() does all the updates to state variables and calculates the metric,

result() returns the value for the metric from state variables.

reset_state() sets the metric value at the beginning of each epoch to a predefined constant.

6. How to get the neural network weights for *every epoch* using Keras callbacks?

Problem

In the code below there is a simple model. The goal is to get weights for every epoch from the model after it is trained. How to do it using *callbacks*?

```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from keras import layers


# Set seed to get reproducible results
np.random.seed(42)
tf.random.set_seed(42)

# Generate data
start, stop = 1,100
cnt = stop - start + 1
b,k = 1,2

xs = np.linspace(start, stop, num = cnt)
ys = np.array([k*x + b for x in xs])

# Simple model with one feature and one unit for regression
task
model = keras.Sequential([
       layers.Dense(units=1, input_shape=[1],
activation='relu')
])
model.compile(loss='mae', optimizer='adam')

batch_size = 20
epochs = 10

# Write callback
...
```

Answer

You may use the Lambda callback and get_weights() method from the Sequential model to hand save the weights in a dictionary.

```
# Generate Callback
weights_dict = {}
weight_callback = tf.keras.callbacks.LambdaCallback \
( on_epoch_end=lambda epoch, logs:
weights_dict.update({epoch:model.get_weights()}))


history = model.fit(xs, ys, batch_size=batch_size,
epochs=epochs,
                    callbacks=[weight_callback],verbose=0)

# Retrive weights
for epoch,weights in weights_dict.items():
  print(f"Epoch #{epoch}")
  print(f"Weights: {weights[0]}, bias: {weights[1]} \n")
```

```
Epoch #0
Weights: [[0.5750251]], bias: [0.00499998]

Epoch #1
Weights: [[0.57998466]], bias: [0.00999998]

Epoch #2
Weights: [[0.58493966]], bias: [0.01499997]

Epoch #3
Weights: [[0.5898572]], bias: [0.01999997]

Epoch #4
Weights: [[0.5948121]], bias: [0.02499996]

Epoch #5
Weights: [[0.5997918]], bias: [0.02999996]

Epoch #6
Weights: [[0.6047719]], bias: [0.03499996]

Epoch #7
Weights: [[0.6097574]], bias: [0.03999995]

Epoch #8
Weights: [[0.61469406]], bias: [0.04499994]

Epoch #9
Weights: [[0.6196337]], bias: [0.04999993]
```

7. Can you give an example to create a tensor using the constant() function in TensorFlow?

Tensors are most commonly created using the constant() function. The values to be input into the tensor are given as arguments as shown below:

```
import tensorflow as tf

t1 = tf.constant([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

t2 = tf.constant(["String One", "String Two", "String Three"])

sess = tf.Session()

print(t1)

print(sess.run(t1))

print("\n")

print(t2)

print(sess.run(t2))
```

8. What is the code that is used to check the version of TensorFlow using Python?

There are two commands depending on the Python version:

Python 2:

```
python -c 'import tensor flow as tf; print(tf.__version__)'
```

Python 3:

```
python3 -c 'import tensor flow as tf; print(tf.__version__)'
```

9. What is the simple syntax that can be used to convert a NumPy array into a tensor?

There are two ways a NumPy array can be converted into a tensor when working with Python. The first one is as follows:

train.shuffle_batch()

And the second way is:

convert_to_tensor(tensor1d, dtype = tf.float64)

The high-level code offers a good amount of readability and ease-of-use and denoted by the above piece of code.